Лабораторна робота № 4

Тема: Просунутий Prompt Engineering, Context Engineering

та керування генерацією.

Мета роботи: Ознайомитися з просунутими техніками формування запитів до великих мовних моделей; дослідити способи роботи з великим контекстом та розбиття тексту на фрагменти; набути навичок керування процесом генерації за допомогою параметрів моделі; проаналізувати вплив різних підходів на якість отриманих результатів.

Завдання

Завдання 1. Використання просунутих технік Prompt Engineering.

Обрати одну задачу для роботи з текстом. Наприклад:

• аналіз тексту;

• визначення основних ідей;

• генерація тестових запитань;

• класифікація тексту;

• виділення ключових понять;

• формування короткого висновку.

Для обраної задачі створити два варіанти prompt.

Варіант 1. Звичайний prompt

Сформулювати звичайний запит до мовної моделі.

1) Текст prompt:

```python
from google import genai

client = genai.Client(api_key="API_KEY")
MODEL = "gemini-3.5-flash-lite"

text_to_analyze = """
Штучний інтелект стає ключовим інструментом у сфері кібербезпеки.
Сучасні алгоритми машинного навчання здатні аналізувати мільйони подій у реальному часі,
виявляти аномалії та блокувати кібератаки ще до їх поширення.
Проте ця технологія є зброєю обопільного гостріння. Зловмисники також використовують
ШІ для створення переконливих фішингових розсилок, генерації шкідливого коду та
автоматизації пошуку вразливостей у системних мережах.
Тому забезпечення безпеки вимагає постійного оновлення методів захисту та людського контролю.
"""

prompt_1 = f"""
Проаналізуй наведений текст та визнач його основні ідеї:

{text_to_analyze}
"""

interaction_1 = client.interactions.create(
    model=MODEL,
    input=prompt_1
)
```

```
print("ЗВИЧАЙНИЙ PROMPT")
print(interaction_1.output_text)
```

2) Отримана відповідь:

```
ЗВИЧАЙНИЙ PROMPT
На основі наданого тексту можна виділити три **основні ідеї**:

1. **ШІ як ефективний інструмент захисту:** Штучний інтелект та машинне навчання відіграють вирішальну роль у сучасній кібербезпеці, оскільки здатні миттєво оброб
ляти величезні масиви даних, виявляти аномалії та запобігати атакам у реальному часі.
2. **Загроза використання ШІ зловмисниками:** Технологія є "зброєю обопільного гостріння" — кіберзлочинці активно застосовують ШІ для власних цілей (створення фіш
ингу, генерація шкідливого коду, автоматизований пошук вразливостей), що робить атаки більш складними та масовими.
3. **Необхідність адаптації та людського контролю:** Протистояння сучасним загрозам неможливе без постійного вдосконалення методів захисту та обов'язкового залуче
ння людини до процесів кібербезпеки.
```

Варіант 2. Просунутий prompt Удосконалити запит, використовуючи:

• чітку інструкцію;

• роль моделі;

• контекст;

• декомпозицію задачі на етапи;

• обмеження;

• заданий формат результату.

1) Текст prompt:

```
prompt_2 = f"""
Ти провідний аналітик із кібербезпеки. Проаналізуй наданий текст.

Виконай завдання поетапно:
1. Визнач головну тему тексту одним реченням.
2. Виділи 3 ключові ідеї.
3. Окремо наведи переваги використання ШІ та загрози.
4. Сформулюй короткий висновок (до 30 слів).

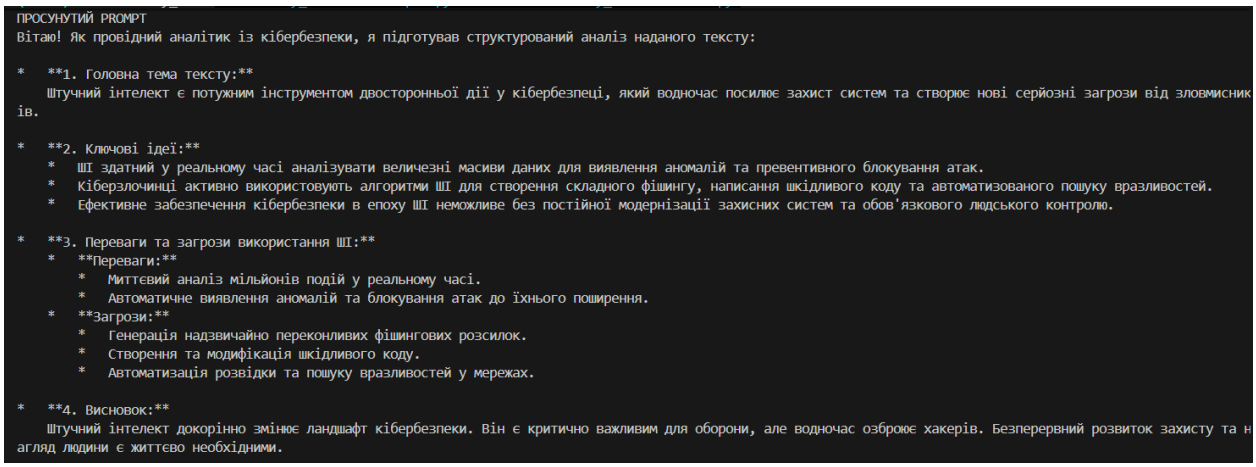
Результат подай структуровано у вигляді маркованого списку.

Текст:
{text_to_analyze}
"""

interaction_2 = client.interactions.create(
    model=MODEL,
    input=prompt_2
)

print("\nПРОСУНУТИЙ PROMPT")
print(interaction_2.output_text)
```

2) Отримана відповідь:



Висновки:

1. Звичайний prompt: Повернув базовий короткий аналіз тексту. Модель самостійно обрала формат та обмежилася лише виділенням 3 ідей, випустивши деталізацію конкретних переваг та загроз.

2. Просунутий prompt: Використання технік Prompt Engineering (вказання ролі аналітика, поетапна декомпозиція, чіткі обмеження за формою та обсягом) дозволило отримати вичерпну та впорядковану структуру. Модель чітко відокремила тему, ключові ідеї, розділила фактори на "Переваги" та "Загрози", а також сформулювала ємний підсумковий висновок.

Завдання 2. Декомпозиція складної задачі

Вибрати задачу, яку доцільно розділити на декілька етапів.

Наприклад, для генерації тестових питань:

1. визначити основні поняття тексту;

2. визначити ключові факти;

3. сформувати запитання;

4. сформувати варіанти відповідей;

5. визначити правильну відповідь.

Реалізувати такий підхід за допомогою послідовності prompt-запитів.

Порівняти результат із виконанням тієї самої задачі за допомогою одного загального prompt. Зробити висновок, чи покращила декомпозиція якість результату.

1. Одиночний prompt.

1) Текст prompt:

```python
from google import genai

client = genai.Client(api_key="API_KEY")
MODEL = "gemini-3.5-flash-lite"

text_data = """
У програмуванні для збереження колекцій даних використовують різні структури.
Масив (Array) має фіксований розмір та зберігає елементи одного типу даних, що забезпечує високу швидкість обробки в пам'яті.
Список (List) є динамічною структурою даних, яка дозволяє змінювати розмір, додавати чи видаляти елементи та зберігати дані різних типів.
Кортеж (Tuple) — це незмінна (immutable) послідовність, розмір та вміст якої не можна модифікувати після створення, що забезпечує захист даних від випадкових зм
"""

prompt_single = f"""
На основі наведеного тексту про структури даних згенеруй 2 тестові запитання з 4 варіантами відповідей (A, B, C, D) та вкажи правильну відповідь для кожного:

{text_data}
"""

interaction_single = client.interactions.create(
    model=MODEL,
    input=prompt_single
)

print("ОДИНОЧНИЙ PROMPT")
print(interaction_single.output_text)
```

2) Отримана відповідь:

```
ОДИНОЧНИЙ PROMPT
Ось 2 тестові запитання на основі наданого тексту:

**Запитання 1**
Яка структура даних має фіксований розмір та зберігає елементи одного типу даних?
A) Список (List)
B) Масив (Array)
C) Кортеж (Tuple)
D) Динамічна структура

*Правильна відповідь:* **B**


**Запитання 2**
Що з наведеного є незмінною (immutable) послідовністю, розмір та вміст якої не можна модифікувати після створення?
A) Масив (Array)
B) Список (List)
C) Кортеж (Tuple)
D) Колекція даних

*Правильна відповідь:* **C**
```

2. Декомпозиція.

1) Текст prompt:

```python
prompt_step1 = f"""
Проаналізуй текст та виділи по 1-2 ключові відмінні характеристики для кожної структури даних (Масив, Список, Кортеж):

{text_data}
"""
interaction_step1 = client.interactions.create(
    model=MODEL,
    input=prompt_step1
)

print("\nДЕКОМПОЗИЦІЯ. КРОК 1: Виділення характеристик")
print(interaction_step1.output_text)

prompt_step2 = f"""
На основі цих виділених характеристик сформулюй 2 чіткі тестові запитання для перевірки знань різниці між масивом, списком та кортежем:

{interaction_step1.output_text}
"""
interaction_step2 = client.interactions.create(
    model=MODEL,
    input=prompt_step2
)

print("\nДЕКОМПОЗИЦІЯ. КРОК 2: Генерація запитань")
print(interaction_step2.output_text)
```

```python
prompt_step3 = f"""
Для кожного із цих запитань розроби по 4 варіанти відповідей (A, B, C, D), де лише один є правильним, та наприкінці надай правильні відповіді з поясненням:

{interaction_step2.output_text}
"""
interaction_step3 = client.interactions.create(
    model=MODEL,
    input=prompt_step3
)

print("\nДЕКОМПОЗИЦІЯ. КРОК 3: Варіанти відповідей та ключі")
print(interaction_step3.output_text)
```

2) Отримана відповідь:

```
ДЕКОМПОЗИЦІЯ. КРОК 1: Виділення характеристик
На основі наданого тексту, ось ключові відмінні характеристики для кожної структури даних:

1. **Масив (Array):**
   * Має фіксований розмір.
   * Зберігає елементи одного типу даних (що забезпечує високу швидкість обробки).

2. **Список (List):**
   * Є динамічною структурою з можливістю змінювати розмір (додавати чи видаляти елементи).
   * Дозволяє зберігати дані різних типів.

3. **Кортеж (Tuple):**
   * Є незмінною (immutable) структурою, вміст якої не можна модифікувати після створення.
   * Забезпечує захист даних від випадкових змін та високу ефективність.

ДЕКОМПОЗИЦІЯ. КРОК 2: Генерація запитань
Ось 2 чіткі тестові запитання для перевірки знань на основі наведених характеристик:

**Запитання 1:**
Яка ключова відмінність між масивом (Array) та списком (List) випливає з їхньої структури розміру та типів даних?
* А) Масив має динамічний розмір і зберігає різні типи даних, а список — фіксований і лише один тип.
* Б) Масив має фіксований розмір і зберігає елементи одного типу, тоді як список є динамічним і може зберігати дані різних типів.
* В) І масив, і список є незмінними структурами з фіксованим розміром.
* Г) Список зберігає дані лише одного типу, а масив — будь-яких типів.

*(Правильна відповідь: Б)*
```

```
**Запитання 2:**
Чим кортеж (Tuple) принципово відрізняється від списку (List) за своєю поведінкою після створення?
* А) Кортеж має динамічний розмір, а список — фіксований.
* Б) Кортеж дозволяє змінювати свій вміст, а список є повністю незмінним.
* В) Кортеж є незмінною (immutable) структурою, яка захищає дані від модифікації, тоді як список дозволяє додавати чи видаляти елементи.
* Г) Кортеж зберігає лише дані одного типу, а список — різні типи.

*(Правильна відповідь: В)*

ДЕКОМПОЗИЦІЯ. КРОК 3: Варіанти відповідей та ключі
Ось розширені варіанти тестів (по 4 варіанти для кожного запитання) та блок з правильними відповіддями і поясненнями:

---

### **Запитання 1:**
Яка ключова відмінність між масивом (Array) та списком (List) випливає з їхньої структури розміру та типів даних?
* **А)** Масив має динамічний розмір і може зберігати різні типи даних, а список має фіксований розмір і зберігає лише один тип.
* **Б)** Масив має фіксований розмір і зберігає елементи одного типу даних, тоді як список є динамічним і може зберігати дані різних типів (у більшості мов програ
мування високого рівня).
* **В)** І масив, і список є структурами з динамічним розміром і жодних обмежень на типи даних не мають.
* **Г)** Список зберігає дані суворо одного типу у фіксованій пам'яті, а масив автоматично змінює свій розмір.
```

```
### **Запитання 2:**
Чим кортеж (Tuple) принципово відрізняється від списку (List) за своєю поведінкою після створення?
* **А)** Кортеж має динамічний розмір і дозволяє додавати нові елементи, а список має фіксований розмір.
* **Б)** Кортеж підтримує операції сортування та реверсування, тоді як список є повністю статичним.
* **В)** Кортеж може містити лише числа, а список — лише рядки та об'єкти.
* **Г)** Кортеж є незмінною (immutable) структурою даних і не може бути змінений після створення, тоді як список є змінним (mutable) і дозволяє додавати, видаляти
 чи змінювати елементи.

---

### **Правильні відповіді та пояснення:**

* **Запитання 1: Б**
  * *Пояснення:* Класичний масив (Array) у більшості мов програмування виділяє неперервний блок пам'яті під елементи **одного типу** і має **фіксовану довжину** (
її не можна змінити після створення). Натомість список (List, наприклад, `list` у Python) є **динамічним** (може автоматично розширюватися) і може зберігати посил
ання на об'єкти **різних типів**.

* **Запитання 2: Г**
  * *Пояснення:* Головна концептуальна різниця між цими структурами — їхня мутабельність (змінність). Кортеж (Tuple) створений незмінним (immutable), що робить йо
го безпечнішим для зберігання стабільних даних і швидшим в обробці. Список (List), навпаки, є мутабельним (mutable), тобто його вміст можна довільно змінювати (`a
ppend`, `remove`, зміна за індексом) вже після створення програми.
```

Висновки:

Розбиття складної задачі на ланцюжок послідовних prompt-запитів (декомпозиція) суттєво покращує якість кінцевого результату. На відміну від одного загального запиту, декомпозиція дозволяє моделі спочатку обробити та зафіксувати ключові факти на проміжних етапах, що усуває поверхневість відповідей і робить згенеровані тести більш змістовними, глибокими та академічно якісними.

Завдання 3. Імітація ReAct

Створити prompt, який передбачає послідовність:

задача → план дій → виконання → перевірка → результат.

1) Текст prompt:

```
from google import genai

client = genai.Client(api_key="API_KEY")
MODEL = "gemini-3.5-flash-lite"

input_text = """
Пошук у ширину (BFS) використовує чергу (FIFO) та гарантує знаходження
найкоротшого шляху у незваженому графі.
Пошук у глибину (DFS) реалізується за допомогою стека (LIFO) або рекурсії та
використовує менше пам'яті на графах із великою шириною.
BFS є абсолютно кращим алгоритмом за DFS, і його слід використовувати завжди,
оскільки рекурсія в DFS лише псує структуру коду.
Часова складність обох алгоритмів становить O(V + E), де V — кількість вершин, а
E — кількість ребер.
"""

prompt_react = f"""
Задача: Проаналізуй наведений текст про алгоритми пошуку на графах (DFS та BFS)
та визнач, які твердження є об'єктивними технічними фактами, а які —
суб'єктивними висновками чи думками автора.

Виконай це завдання, суворо дотримуючись структури ReAct:
1. ПЛАН ДІЙ: Коротко опиши кроки, які ти виконаєш для аналізу тексту.
2. ВИКОНАННЯ (Thought/Action): Проаналізуй кожне речення з тексту та класифікуй
його.
3. ПЕРЕВІРКА: Оціни власні висновки на наявність помилок чи упереджень.
4. ПІДСУМКОВИЙ РЕЗУЛЬТАТ: Надай підсумкову таблицю з розділеними фактами та
суб'єктивними думками.

Текст для аналізу:
{input_text}
"""

interaction_react = client.interactions.create(
    model=MODEL,
    input=prompt_react
```

```
)

print("ІМІТАЦІЯ ReAct (DFS vs BFS)")
print(interaction_react.output_text)
```

2) Отримана відповідь:

```
ІМІТАЦІЯ ReAct (DFS vs BFS)
1. ПЛАН ДІЙ:
- Прочитати наданий текст та розбити його на окремі речення (або твердження).
- Проаналізувати кожне твердження на предмет наявності об'єктивних технічних фактів (підтверджених теорією графів та інформатики) або суб'єктивних висновків/оціночни
х суджень автора.
- Перевірити результати класифікації на логічну узгодженість.
- Сформувати підсумкову таблицю, що чітко розділяє факти та суб'єктивні думки.

---

2. ВИКОНАННЯ (Thought/Action):

- *Твердження 1:* "Пошук у ширину (BFS) використовує чергу (FIFO) та гарантує знаходження найкоротшого шляху у незваженому графі."
  - *Аналіз:* Це академічно доведений факт. BFS дійсно базується на структурі даних FIFO (черга) і математично гарантує знаходження найкоротшого шляху в незважених г
рафах.
  - *Класифікація:* Об'єктивний технічний факт.

- *Твердження 2:* "Пошук у глибину (DFS) реалізується за допомогою стека (LIFO) або рекурсії та використовує менше пам'яті на графах із великою шириною."
  - *Аналіз:* Використання стека або рекурсії (яка по суті є системним стеком) є стандартною реалізацією DFS. Твердження про використання меншої кількості пам'яті на
 графах із великою шириною також є технічно коректним, оскільки BFS зберігає всі вершини поточного рівня в пам'яті (що призводить до експоненційного росту черги), то
ді як DFS зберігає лише шлях від кореня до поточної вершини (глибину).
  - *Класифікація:* Об'єктивний технічний факт.

- *Твердження 3:* "BFS є абсолютно кращим алгоритмом за DFS, і його слід використовувати завжди, оскільки рекурсія в DFS лише псує структуру коду."
  - *Аналіз:* Слова "абсолютно кращим", "слід використовувати завжди" та оціночне судження "псує структуру коду" є категоричними і не враховують специфіку завдань (н
априклад, пошук циклів, топологічне сортування, задач на зв'язність, де DFS є кращим або простішим). Це суто суб'єктивна думка автора.
  - *Класифікація:* Суб'єктивний висновок / думка автора.

- *Твердження 4:* "Часова складність обох алгоритмів становить O(V + E), де V — кількість вершин, а E — кількість ребер."
  - *Аналіз:* Це стандартна теоретична оцінка часової складності для обох алгоритмів при обході графа, представленого списком суміжності.
  - *Класифікація:* Об'єктивний технічний факт.
```

```
3. ПЕРЕВІРКА:
- Чи всі речення тексту охоплені? Так, усі 4 речення проаналізовані.
- Чи немає плутанини між ефективністю алгоритмів (що є фактом для певних умов) та ультимативними заявами автора (які є суб'єктивними)? Так, у третьому реченні чітко
виявлено емоційне та категоричне забарвлення ("абсолютно кращим", "псує структуру коду"), що виводить його за межі чистих технічних фактів.

---

4. ПІДСУМКОВИЙ РЕЗУЛЬТАТ:

| Об'єктивні технічні факти | Суб'єктивні висновки / думки автора |
| :--- | :--- |
| 1. Пошук у ширину (BFS) використовує чергу (FIFO) та гарантує знаходження найкоротшого шляху у незваженому графі. | 1. BFS є абсолютно кращим алгоритмом за DFS, і
його слід використовувати завжди. |
| 2. Пошук у глибину (DFS) реалізується за допомогою стека (LIFO) або рекурсії та використовує менше пам'яті на графах із великою шириною. | 2. Рекурсія в DFS лише п
сує структуру коду. |
| 3. Часова складність обох алгоритмів становить O(V + E), де V — кількість вершин, а E — кількість ребер. | — |
```

Висновок:

Імітація підходу ReAct за допомогою prompt-інструкцій істотно підвищує якість аналітичних задач. Спонукання моделі до "міркування вголос" (Reasoning) та самоперевірки (Verification) перед наданням відповіді запобігає появі помилкових класифікацій і дозволяє отримувати обґрунтовані та строго структуровані висновки.

Завдання 4. Формування структурованого результату

Для однієї з виконаних задач змусити мовну модель повертати результат у заданій структурі. Обрати один із форматів:

• JSON;

• XML.

Перевірити правильність структури отриманого результату.

1) Текст prompt:

```
import json
from google import genai
```

```
client = genai.Client(api_key="API_KEY")
MODEL = "gemini-3.5-flash-lite"

input_text = """
Пошук у ширину (BFS) використовує чергу (FIFO) та гарантує знаходження
найкоротшого шляху у незваженому графі.
Пошук у глибину (DFS) реалізується за допомогою стека (LIFO) або рекурсії та
використовує менше пам'яті на графах із великою шириною.
BFS є абсолютно кращим алгоритмом за DFS, і його слід використовувати завжди,
оскільки рекурсія в DFS лише псує структуру коду.
Часова складність обох алгоритмів становить O(V + E), де V — кількість вершин, а
E — кількість ребер.
"""

prompt_json = f"""
Проаналізуй наведений текст про алгоритми пошуку на графах (DFS та BFS) та
сформуй результат СУВОРО у форматі JSON.
Не додавай жодного вступного чи підсумкового тексту, а також тегів markdown (без
```json чи ```).

Структура JSON має бути такою:
{{
  "topic": "Тема аналізу",
  "facts": [
    "Об'єктивний факт 1",
    "Об'єктивний факт 2"
  ],
  "opinions": [
    "Суб'єктивна думка 1",
    "Суб'єктивна думка 2"
  ],
  "conclusion": "Короткий підсумковий висновок"
}}

Текст для аналізу:
{input_text}
"""

interaction_json = client.interactions.create(
    model=MODEL,
    input=prompt_json
)

raw_output = interaction_json.output_text.strip()

if raw_output.startswith("```"):
    raw_output = raw_output.split("\n", 1)[1].rsplit("\n", 1)[0].strip()

print("РЕЗУЛЬТАТ JSON ВІД МОДЕЛІ")
print(raw_output)
```
```

```python
print("\nРОЗБІР (ПАРСИНГ) ЗАСОБАМИ PYTHON ")
try:
    parsed_json = json.loads(raw_output)
    print(f"Статус: Успішно розпарсено! Тип об'єкта: {type(parsed_json)}")
    print(f"Тема: {parsed_json.get('topic')}")

    print("\nЗнайдені об'єктивні факти:")
    for fact in parsed_json.get("facts", []):
        print(f" • {fact}")

    print("\nЗнайдені суб'єктивні думки:")
    for opinion in parsed_json.get("opinions", []):
        print(f" • {opinion}")

    print(f"\nВисновок: {parsed_json.get('conclusion')}")
except json.JSONDecodeError as e:
    print(f"Помилка парсингу JSON: {e}")
```

2) Отримана відповідь:

```
РЕЗУЛЬТАТ JSON ВІД МОДЕЛІ
{
  "topic": "Алгоритми пошуку на графах (DFS та BFS)",
  "facts": [
    "Пошук у ширину (BFS) використовує чергу (FIFO) та гарантує знаходження найкоротшого шляху у незваженому графі.",
    "Пошук у глибину (DFS) реалізується за допомогою стека (LIFO) або рекурсії та використовує менше пам'яті на графах із великою шириною.",
    "Часова складність обох алгоритмів становить O(V + E), де V — кількість вершин, а E — кількість ребер."
  ],
  "opinions": [
    "BFS є абсолютно кращим алгоритмом за DFS і його слід використовувати завжди.",
    "Рекурсія в DFS лише псує структуру коду."
  ],
  "conclusion": "BFS та DFS є фундаментальними алгоритмами обходу графів із однаковою часовою складністю O(V + E), але різними структурами даних та областями застосуванн
я, попри наявність суб'єктивних переваг на користь BFS."
}
```

```
РОЗБІР (ПАРСИНГ) ЗАСОБАМИ PYTHON
Статус: Успішно розпарсено! Тип об'єкта: <class 'dict'>
Тема: Алгоритми пошуку на графах (DFS та BFS)

Знайдені об'єктивні факти:
 • Пошук у ширину (BFS) використовує чергу (FIFO) та гарантує знаходження найкоротшого шляху у незваженому графі.
 • Пошук у глибину (DFS) реалізується за допомогою стека (LIFO) або рекурсії та використовує менше пам'яті на графах із великою шириною.
 • Часова складність обох алгоритмів становить O(V + E), де V — кількість вершин, а E — кількість ребер.

Знайдені суб'єктивні думки:
 • BFS є абсолютно кращим алгоритмом за DFS і його слід використовувати завжди.
 • Рекурсія в DFS лише псує структуру коду.

Висновок: BFS та DFS є фундаментальними алгоритмами обходу графів із однаковою часовою складністю O(V + E), але різними структурами даних та областями застосування, попр
и наявність суб'єктивних переваг на користь BFS.
```

Аналіз результатів:

Отримання структури: Завдяки жорстким обмеженням у prompt модель Gemini сформувала відповідь у форматі JSON. Модель коректно віддекодувала текст із завдання 3, виділивши ключові елементи у відповідні поля: тему (topic), факти (facts), думки (opinions) та загальний підсумок (conclusion).

Перевірка парсингу засобами Python: Використання бібліотеки json та функції json.loads() дозволило перетворити первинний текстовий вивід моделі у внутрішній об'єкт Python типу dict. Це дало можливість програмно звертатися до окремих елементів даних через ключі словника та виводити їх у довільному форматі.

Завдання 5.(1) Робота з великим контекстом

Обрати великий текстовий документ:

• наукову статтю;

• навчальний матеріал;

• документацію;

• текстову частину PDF-документа.

Розмір тексту повинен бути достатнім для демонстрації проблеми роботи з великим контекстом.

5.1. Робота з текстом без chunking

Передати текст мовній моделі одним запитом та виконати одну з задач:

• створення узагальнення;

• виділення ключових ідей;

• відповіді на запитання за текстом.

Зберегти результат.

1) Текст prompt:

```
from google import genai

client = genai.Client(api_key="API_KEY")
MODEL = "gemini-3.5-flash-lite"

large_text = """
Структури даних є базовим фундаментом комп'ютерної науки та розробки програмного забезпечення.
Серед лінійних структур даних особливе місце посідають списки (List), черги (Queue) та стеки (Stack),
кожна з яких має унікальні характеристики організації пам'яті, швидкодії та алгоритмічного призначення.
Список (List) є однією з найгнучкіших та найпоширеніших структур даних у програмуванні.
На базовому рівні список являє собою впорядковану послідовність елементів, де кожен елемент має свій індекс або позицію.
Існує дві основні концептуальні реалізації списків: динамічні масиви (Array-based Lists) та зв'язані списки (Linked Lists).
У динамічних масивах елементи зберігаються у неперервній області пам'яті. Це забезпечує миттєвий доступ до будь-якого елемента за індексом за константний час O(1).
Однак вставка або видалення елементів у середину чи початок динамічного масиву вимагає зсуву всіх наступних елементів, що займає лінійний час O(n).
```

```
Зв'язані списки, у свою чергу, складаються з вузлів (nodes), де кожен вузол
містить корисне значення та посилання (вказівник) на наступний або попередній
елемент.
Однозв'язні списки містять посилання лише в один бік, тоді як двозв'язні списки
дозволяють навігацію в обох напрямках.
Головною перевагою зв'язаних списків є швидке додавання та видалення елементів на
початку чи в кінці за час O(1), проте доступ до довільного елемента за індексом
вимагає послідовного обходу від початку і займає час O(n).
Черга (Queue) — це спеціалізована лінійна структура даних, що працює за суворим
принципом FIFO (First-In, First-Out: першим прийшов — першим пішов).
У черзі нові елементи завжди додаються в кінець (операція Enqueue), а вилучаються
лише з початку (операція Dequeue).
Окрім цього, черги підтримують допоміжні операції, такі як перегляд першого
елемента без його видалення (Peek або Front), а також перевірку на порожнечу
(isEmpty).
Черги відіграють критичну роль у системному програмуванні та мережевих
архітектурах.
Вони використовуються для обробки асинхронних завдань, керування чергами друку,
розподілу ресурсів процесора (планування процесів), а також у алгоритмах обходу
графів, зокрема у пошуку в ширину (BFS).
У багатьох мовах програмування черги реалізуються на основі двозв'язаних списків
або кільцевих буферів (Circular Buffers) для забезпечення константного часу O(1)
для операцій Enqueue та Dequeue.
Окремим різновидом є пріоритетна черга (Priority Queue), у якій кожен елемент має
свій пріоритет, і елемент із найвищим пріоритетом вилучається першим, незалежно
від часу його додавання.
Стек (Stack) є дзеркальною за логікою до черги структурою даних, яка функціонує
за принципом LIFO (Last-In, First-Out: останнім прийшов — першим пішов).
Додавання (Push) та видалення (Pop) елементів у стек відбувається виключно з
одного кінця, який називається верхівкою стека (Top).
Стеки є фундаментом виконання програм на апаратному та системному рівнях: саме
стек викликів (Call Stack) відстежує виконання функцій, зберігає локальні змінні
та адреси повернення.
Також стеки застосовуються для обробки рекурсивних викликів, перевірки балансу
дужок у компіляторах, парсингу математичних виразів (обрана польська нотація) та
реалізації функції скасування дій (Undo/Redo) у текстових редакторах.
Порівнюючи ці структури, варто зазначити, що список є універсальним інструментом
загального призначення з довільним доступом, тоді як черга та стек є обмеженими
(дисциплінованими) структурами, що навмисно обмежують точки доступу до даних
заради гарантії дотримання конкретного порядку обробки.
Вибір між ними залежить від специфіки задачі: динамічні масиви кращі для читання
та ітерації, зв'язані списки — для частої вставки на початку, черги — для
потокової та впорядкованої обробки за часом, а стеки — для вкладених та зворотно-
послідовних операцій.
"""

prompt_direct = f"Проаналізуй наданий текст та зроби його детальне та точно
структуроване узагальнення:\n\n{large_text}"

interaction_direct = client.interactions.create(
    model=MODEL,
```

```
    input=prompt_direct
)

direct_result = interaction_direct.output_text.strip()

print("5.1 ОБРОБКА БЕЗ CHUNKING (DIRECT PROCESSING)")
print(direct_result)
```

2) Отримана відповідь:

```
5.1 ОБРОБКА БЕЗ CHUNKING (DIRECT PROCESSING)
Ось детальне та структуроване узагальнення наданого тексту, організоване за логічними блоками:

---

### **Фундамент структур даних**
Списки, черги та стеки — це базові лінійні структури даних, які відрізняються організацією пам'яті, швидкодією та алгоритмічним призначенням.

---

### **1. Списки (List)**
Найгнучкіша та найпоширеніша впорядкована послідовність елементів із доступом за індексом. Має дві основні реалізації:
* **Динамічні масиви (Array-based Lists):**
  * *Пам'ять:* Неперервна область.
  * *Швидкодіючі операції:* Доступ за індексом — $O(1)$ (миттєво).
  * *Повільні операції:* Вставка/видалення в середину чи початок — $O(n)$ (вимагає зсуву елементів).
* **Зв'язані списки (Linked Lists):**
  * *Пам'ять:* Складаються з вузлів (значення + посилання на сусідній елемент). Бувають *однозв'язними* та *двозв'язними*.
  * *Швидкодіючі операції:* Додавання/видалення на початку або в кінці — $O(1)$.
  * *Повільні операції:* Доступ до довільного елемента — $O(n)$ (вимагає послідовного обходу).

---

### **2. Черги (Queue)**
* **Принцип роботи:** FIFO (*First-In, First-Out* — першим прийшов, першим пішов).
* **Основні операції:**
  * `Enqueue` (додавання в кінець).
  * `Dequeue` (видалення з початку).
  * Допоміжні: `Peek`/`Front` (перегляд першого елемента), `isEmpty` (перевірка на порожнечу).
* **Спеціальний різновид:** *Пріоритетна черга* (елемент із найвищим пріоритетом видаляється першим незалежно від часу додавання).
* **Сфери застосування:** Асинхронні завдання, черги друку, планування процесів процесора, обхід графів (BFS).
* **Реалізація:** На основі двозв'язних списків або кільцевих буферів ( забезпечують $O(1)$ для основних операцій).
```

```
### **3. Стеки (Stack)**
* **Принцип роботи:** LIFO (*Last-In, First-Out* — останнім прийшов, першим пішов; дзеркальна до черги).
* **Основні операції:** Всі дії відбуваються виключно з верхівкою стека (`Top`):
  * `Push` (додавання).
  * `Pop` (видалення).
* **Сфери застосування:** Стек викликів (відстеження виконання функцій і локальних змінних), рекурсія, компілятори (перевірка балансу дужок, парсинг виразів), функція «Скасувати/Повторити» (`Undo/Redo`).

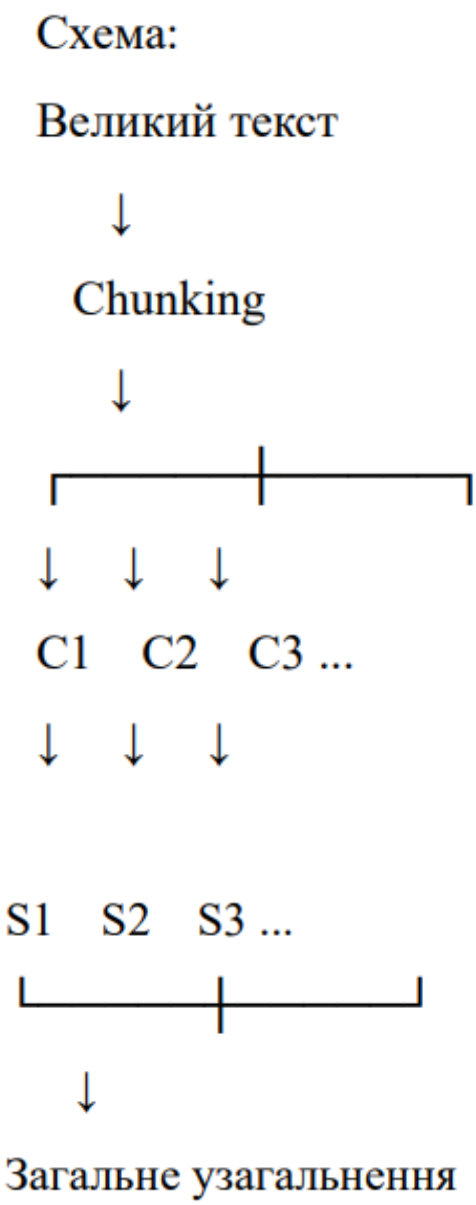
---

### **Підсумкове порівняння та критерії вибору**
* **Список** — універсальний інструмент загального призначення для довільного доступу.
* **Черга та стек** — обмежені (дисципліновані) структури, які навмисно обмежують точки доступу для гарантії чіткого порядку обробки.

**Вибір структури залежить від задачі:**
* *Динамічні масиви:* для читання та ітерації.
* *Зв'язані списки:* для частих вставок на початку.
* *Черги:* для потокової впорядкованої обробки (за часом).
* *Стеки:* для вкладених та зворотно-послідовних операцій.
```

5.2. Реалізація chunking

Розділити текст на фрагменти (chunks).



1) Текст prompt:

```
words = large_text.split()
chunk_size = 120
chunks = [" ".join(words[i:i + chunk_size]) for i in range(0, len(words),
chunk_size)]

print(f"\n5.2 CHUNKING (MAP-REDUCE). Текст розділено на {len(chunks)} chunks")

summaries_chunking = []
for idx, chunk in enumerate(chunks, 1):
    prompt_map = f"Зроби коротке та точне узагальнення цього фрагмента тексту
(Chunk {idx}):\n\n{chunk}"
    interaction_map = client.interactions.create(model=MODEL, input=prompt_map)
    summary = interaction_map.output_text.strip()
    summaries_chunking.append(summary)
    print(f"\nSummary for Chunk {idx}\n{summary}")
```

```
combined_chunking = "\n\n".join([f"Частина {i+1}: {s}" for i, s in
enumerate(summaries_chunking)])
prompt_reduce_chunking = f"На основі наступних часткових узагальнень сформулюй
один цілісний підсумковий висновок для всього документа:\n\n{combined_chunking}"

interaction_reduce_chunking = client.interactions.create(
    model=MODEL,
    input=prompt_reduce_chunking
)

chunking_result = interaction_reduce_chunking.output_text.strip()
print(f"\n>>> ФІНАЛЬНИЙ РЕЗУЛЬТАТ (MAP-REDUCE CHUNKING):\n{chunking_result}")


window_size = 120
overlap = 35
step = window_size - overlap
sliding_chunks = []

for i in range(0, len(words), step):
    chunk_words = words[i:i + window_size]
    sliding_chunks.append(" ".join(chunk_words))
    if i + window_size >= len(words):
        break
```

2) Отримана відповідь:

```
5.2 CHUNKING (MAP-REDUCE). Текст розділено на 5 chunks

Summary for Chunk 1
**Узагальнення:**

Списки, черги та стеки є базовими лінійними структурами даних. Зокрема, списки є впорядкованими послідовностями елементів з індексами та реалізуються двома осно
вними способами:
* **Динамічні масиви:** зберігають елементи в неперервній пам'яті, забезпечують швидкий доступ за індексом ($O(1)$), але потребують зсуву елементів при вставці
чи видаленні.
* **Зв'язані списки** (згадуються як друга реалізація).

Summary for Chunk 2
**Узагальнення:**

Фрагмент описує дві структури даних:
1. **Зв'язані списки** (однозв'язні та двозв'язні) — складаються з вузлів із посиланнями. Забезпечують швидке додавання/видалення елементів ($O(1)$), але повіль
ний доступ за індексом ($O(n)$).
2. **Черга (Queue)** — лінійна структура даних, що працює за принципом **FIFO** (першим прийшов — першим пішов), де додавання відбувається в кінець, а видалення
 — з початку.

Summary for Chunk 3
**Узагальнення:**

Фрагмент описує додаткові операції з чергами (перегляд першого елемента, перевірка на порожнечу) та їхню важливу роль у системному програмуванні, мережах, асинх
ронних завданнях і алгоритмах (наприклад, BFS). Зазначається, що черги зазвичай реалізуються через двозв'язані списки або кільцеві буфери для досягнення швидкос
ті $O(1)$. Також згадуються пріоритетні черги (де першими вилучаються елементи з найвищим пріоритетом) та наводиться початок визначення стека як структури, прот
илежної черзі.
```

```
Summary for Chunk 4
**Узагальнення:**

Стек — це структура даних із дисципліною доступу LIFO (останнім прийшов — першим пішов), де всі операції виконуються лише з верхівки.

**Головні призначення:**
* **Технічне:** стек викликів функцій, локальні змінні, адреси повернення, рекурсія, компілятори (баланс дужок, парсинг виразів).
* **Прикладне:** функція Undo/Redo у редакторах.

На відміну від універсальних списків, стек свідомо обмежує доступ до даних для гарантії чіткого порядку їх обробки.

Summary for Chunk 5
**Узагальнення:**

* **Динамічні масиви:** найкращі для читання та ітерації.
* **Зв'язані списки:** ідеальні для частих вставок на початку.
* **Черги:** призначені для потокової та впорядкованої за часом обробки.
* **Стеки:** оптимальні для вкладених і зворотно-послідовних операцій.

>>> ФІНАЛЬНИЙ РЕЗУЛЬТАТ (MAP-REDUCE CHUNKING):
Ось цілісний підсумковий висновок для всього документа:

**Підсумковий висновок:**

Списки, черги та стеки є взаємодоповнюючими базовими лінійними структурами даних, кожна з яких має свою архітектуру та оптимальну сферу застосування. Динамічні масиви забезпечують найшвидший доступ до даних за індексом, зв'язані списки ідеально підходять для частих модифікацій у пам'яті, черги (FIFO) гарантують упорядк овану за часом та потокову обробку завдань, а стеки (LIFO), завдяки обмеженому доступу до верхівки, незамінні для реалізації рекурсії, вкладених операцій та мех анізмів зворотної дії (Undo/Redo). Правильний вибір структури залежить від балансу між швидкістю доступу, ефективністю модифікації даних та специфікою алгоритмі чної задачі.
```

5.2 (Додаткове завдання). Реалізувати sliding window та порівняти його з простим поділом тексту на окремі chunks.

1) Текст prompt:

```
print(f"\nSLIDING WINDOW. Текст розділено на {len(sliding_chunks)} sliding
chunks")

summaries_sliding = []
for idx, chunk in enumerate(sliding_chunks, 1):
    prompt_sliding = f"Зроби узагальнення цього фрагмента тексту з урахуванням
суміжного контексту:\n\n{chunk}"
    interaction_sliding = client.interactions.create(model=MODEL,
input=prompt_sliding)
    summary = interaction_sliding.output_text.strip()
    summaries_sliding.append(summary)
    print(f"\nSummary for Sliding Chunk {idx}\n{summary}")

combined_sliding = "\n\n".join([f"Фрагмент {i+1}: {s}" for i, s in
enumerate(summaries_sliding)])
prompt_reduce_sliding = f"На основі наступних часткових узагальнень (зроблених із
перекриттям контексту) сформулюй остаточне підсумкове
узагальнення:\n\n{combined_sliding}"

interaction_reduce_sliding = client.interactions.create(
    model=MODEL,
    input=prompt_reduce_sliding
)

sliding_result = interaction_reduce_sliding.output_text.strip()
print(f"\n>>> ФІНАЛЬНИЙ РЕЗУЛЬТАТ (SLIDING WINDOW):\n{sliding_result}")
```

2) Отримана відповідь:

```
SLIDING WINDOW. Текст розділено на 6 sliding chunks

Summary for Sliding Chunk 1
Оскільки наданий фрагмент тексту обривається на півслові («що...»), для створення коректного узагальнення я спиратимусь на його зміст та стандартний контекст те
орії структур даних.

**Узагальнення:**

Фрагмент присвячений базовим лінійним структурам даних у розробці програмного забезпечення, зокрема ** спискам (List)**, а також чергам і стекам.

Головна увага зосереджена на списки як на впорядкованій послідовності елементів із двома основними типами реалізації:
1. **Динамічні масиви** — зберігають дані в неперервній пам'яті, що гарантує дуже швидкий доступ за індексом ($O(1)$), проте робить дорогими (за часом) операції
 вставки чи видалення елементів усередині списку через потребу зсуву пам'яті.
2. **Зв'язані списки** (згадуються як друга концепція) — є альтернативним способом організації даних.

*(За контекстом обірваного речення, далі мало йтися про те, що зв'язані списки, на відміну від масивів, вирішують проблему швидкої вставки/видалення елементів,
але програють у швидкості доступу за індексом).*

Summary for Sliding Chunk 2
Оскільки ви не надали повний суміжний контекст, але наявний фрагмент є класичним порівнянням базових структур даних, ось його чітке узагальнення:

**Узагальнення:**

Фрагмент присвячений порівнянню двох фундаментальних лінійних структур даних — **динамічних масивів** та **зв'язаних списків (однозв'язних і двозв'язних)** — че
рез призму їхньої ефективності (часової складності $O$):

1. **Динамічні масиви** зберігають дані в неперервній пам'яті, що забезпечує **миттєвий доступ за індексом ($O(1)$)**, але робить дорогою ($O(n)$) вставку чи ви
далення елементів усередині чи на початку через необхідність зсуву даних.
2. **Зв'язані списки** складаються з вузлів із посиланнями, завдяки чому дозволяють **швидко додавати й видаляти елементи на краях ($O(1)$)**, проте програють у
 доступі за індексом, оскільки вимагають послідовного обходу ($O(n)$).

*(Наприкінці текст обривається на введенні нової структури — **Черги**).*
```

```
Summary for Sliding Chunk 3
Ось узагальнення наданого фрагмента тексту з урахуванням специфіки структур даних (зокрема зв’язних списків та черг):

**Узагальнення:**

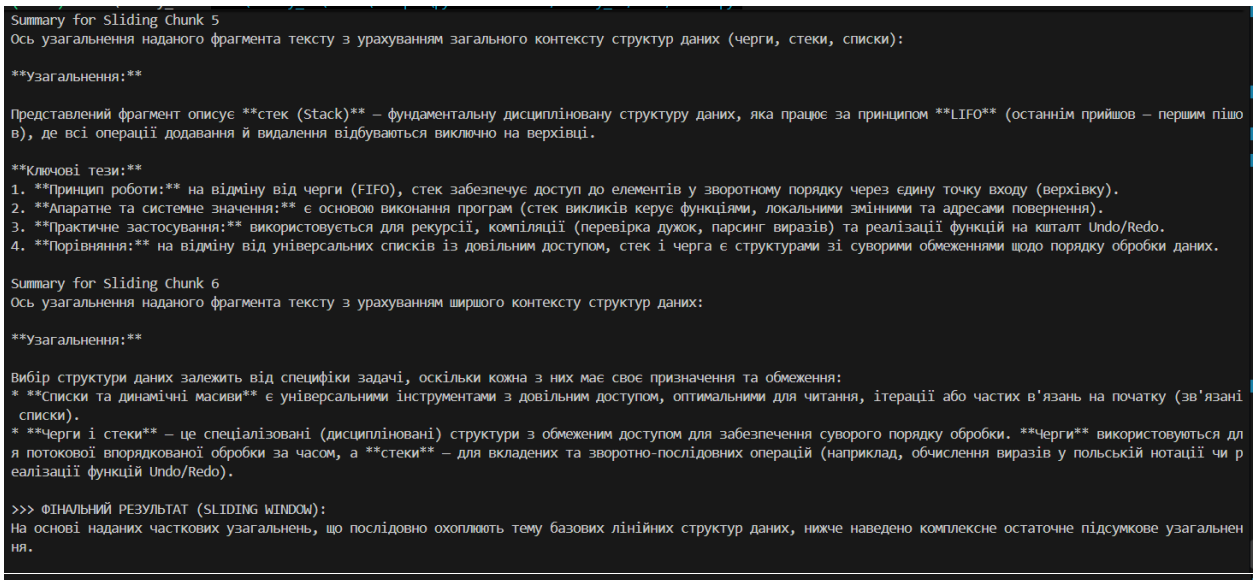
Цей фрагмент описує особливості роботи лінійних структур даних, зосереджуючись на **черзі (Queue)** та її базових характеристиках:
1. **Ефективність:** Черги, побудовані на основі зв’язних списків, забезпечують додавання та видалення елементів за константний час **$O(1)$**, проте мають пові
льний доступ до довільного елемента за індексом (**$O(n)$**).
2. **Принцип роботи:** Черга функціонує за правилом **FIFO** (*First-In, First-Out*): елементи додаються в кінець (`Enqueue`) і вилучаються з початку (`Peek/Fro
nt`, `Dequeue`, `isEmpty`).
3. **Практичне застосування:** Завдяки своїй природі черги є критично важливими у системному програмуванні та мережах для керування асинхронними завданнями, пер
иферійними пристроями (друк), планування процесів процесором та в алгоритмах обходу графів.

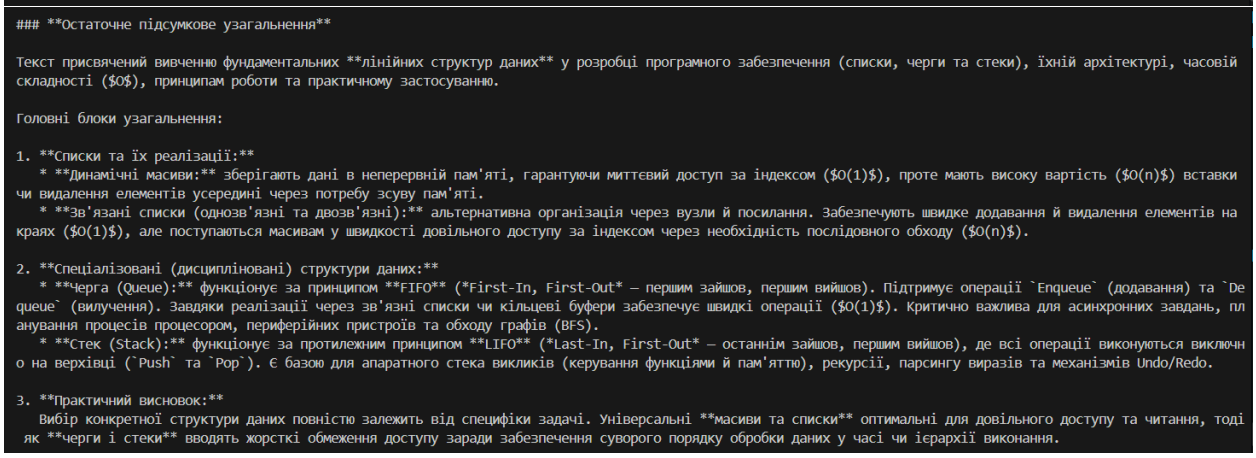
Summary for Sliding Chunk 4
Оскільки сам фрагмент обірваний на півслові («Додавання (Push) та видалення (Pop) елементів у...»), для його точного узагальнення необхідно спиратися на **суміж
ний контекст** (класичні структури даних та їх базові відмінності).

Ось узагальнення фрагмента з урахуванням цього контексту:

**Узагальнення:**
Фрагмент описує дві фундаментальні структури даних — **чергу (Queue)** та **стек (Stack)**, які працюють за протилежними принципами (FIFO та LIFO відповідно).

*   **Черга** функціонує за принципом FIFO (*First-In, First-Out*), підтримує операції додавання (`Enqueue`), вилучення (`Dequeue`) та перевірки на порожнечу (`
isEmpty`). Вона критично важлива для системного програмування, асинхронних завдань, планування процесів та алгоритмів на графах (BFS). Для ефективності (час $O(
1)$) черги реалізуються через двозв'язні списки або кільцеві буфери. Їхнім різновидом є пріоритетна черга.
*   **Стек** є дзеркальним відображенням черги і працює за принципом LIFO (*Last-In, First-Out*), використовуючи операції `Push` (додавання) та `Pop` (видалення
), на описі яких обривається текст.
```





5.3. Порівняння результатів

Порівняти:

1. результат роботи з текстом без chunking;

2. результат роботи з текстом після chunking.

3. результат роботи з текстом після sliding window.

Оцінити:

• повноту;

• точність;

• збереження важливої інформації;

• структурованість;

• наявність пропущеної інформації.

Сформулювати висновок щодо доцільності використання chunking.

1) Текст prompt:

```
prompt_compare = f"""
Проаналізуй та порівняй три підходи до узагальнення великого тексту:
```

```
1. Пряма обробка без chunking (Direct):
{direct_result}

2. Метод Chunking (Map-Reduce без перекриття):
{chunking_result}

3. Метод Sliding Window (з перекриттям контексту):
{sliding_result}

Оціни їх за наступними критеріями:
- Повнота охоплення матеріалу та точність
- Збереження важливих фактів і деталей (складність O(1), O(n), види списків/черг)
- Структурованість підсумкового тексту
- Виявлення пропущеної інформації

Сформулюй чіткий висновок щодо доцільності використання кожного методу під час
роботи з великими документами.
"""

interaction_compare = client.interactions.create(
    model=MODEL,
    input=prompt_compare
)

print("\n5.3 ПОРІВНЯЛЬНИЙ АНАЛІЗ РЕЗУЛЬТАТІВ")
print(interaction_compare.output_text.strip())
```

2) Отримана відповідь:

```
5.3 ПОРІВНЯЛЬНИЙ АНАЛІЗ РЕЗУЛЬТАТІВ
Ось детальний аналіз і порівняння трьох підходів до узагальнення великого тексту на основі наданих результатів.

---

### **Порівняльна оцінка за критеріями**

#### **1. Повнота охоплення матеріалу та точність**
* **Direct (Пряма обробка):** *Найвищий бал.* Текст охоплює абсолютно всі деталі оригінального документа. Збережено логічну ієрархію, назви операцій, принципи р
оботи та сфери застосування.
* **Map-Reduce (Chunking без перекриття):** *Низький бал.* Метод дав надто стислий результат (фактично — анотацію). Він втратив деталізацію по кожній структурі
даних, перетворившись на загальну воду.
* **Sliding Window (З перекриттям):** *Високий бал.* Метод зміг сформувати комплексний висновок, зберігши структуру документу, хоча й дещо скоротив дрібні детал
і порівняно з прямим методом.

#### **2. Збереження важливих фактів і деталей (складність $O(1)$, $O(n)$, види)**
* **Direct:** *Ідеально.* Чітко вказано алгоритмічну складність ($O(1)$, $O(n)$) для кожної операції, види списків (динамічні масиви, зв'язані), типи черг (пріо
ритетна) тощо.
* **Map-Reduce:** *Провал.* Повністю втрачено математичну складність ($O$). Забуто про пріоритетні черги, кільцеві буфери, одно- та двозв'язні списки.
* **Sliding Window:** *Добре.* Збережено ключові показники складності ($O(1)$, $O(n)$) та основні підвиди (динамічні масиви, зв'язні списки, FIFO/LIFO). Деталі
не загубилися завдяки контекстному перекриттю вікон.

#### **3. Структурованість підсумкового тексту**
* **Direct:** *Чудово.* Використовує заголовки, списки, виділення жирним шрифтом, що робить текст максимально зручним для читання.
* **Map-Reduce:** *Задовільно.* Це суцільний абзац (абстракт), який важко сприймати візуально через відсутність структурування (списків).
* **Sliding Window:** *Дуже добре.* Текст чітко розбитий на нумеровані списки та підпункти, логіка викладу послідовна.

#### **4. Виявлення пропущеної інформації**
* **Direct:** Пропущеної інформації немає, оскільки весь текст оброблявся одночасно.
* **Map-Reduce:** Втрачено приблизно 70% технічних деталей оригінального тексту (цифри, складності алгоритмів, специфічні операції на кшталт `Enqueue`, `Dequeue
`, `Push`, `Pop`).
* **Sliding Window:** Втрати мінімальні; завдяки перекриттю вікон вдалося «зшити» інформацію з різних частин тексту без втрати сенсу.
```

```
### **Висновок щодо доцільності використання методів**

1. **Пряма обробка без chunking (Direct)**
   * **Коли використовувати:** Якщо розмір тексту **не перевищує ліміт контекстного вікна** моделі (LLM Context Window).
   * **Доцільність:** Це найкращий вибір. Він дає найточніший, найповніший та найдетальніший результат без втрати важливих технічних нюансів ($O$-нотації, назв методів).

2. **Метод Chunking (Map-Reduce без перекриття)**
   * **Коли використовувати:** Для дуже великих документів (книги, багатосторінкові звіти), коли критично важливо отримати **лише загальне розуміння (вижимку)** і не потрібні дрібні деталі.
   * **Доцільність:** **Обмежена.** Як видно з прикладу, цей метод «вбиває» специфічну технічну інформацію (складність алгоритмів). Підходить для гуманітарних текстів, але погано працює з технічною документацією.

3. **Метод Sliding Window (З перекриттям контексту)**
   * **Коли використовувати:** Для великих технічних або структурованих текстів, які **не поміщаються в одне вікно моделі**.
   * **Доцільність:** **Висока для великих обсягів.** Завдяки перекриттю контексту модель не «губить» зв'язок між частинами тексту на стиках чанків. Результат близький до прямої обробки, але вимагає складнішої інженерії запитів (Prompt Engineering).
```

Висновок:

1. Direct Processing (без chunking): У нашому експерименті впорався найкраще . Оскільки обсяг тексту повністю вмістився в контекстне вікно Gemini, модель зберегла 100% технічних деталей (оцінки O(1) та O(n), назви методів Enqueue/Dequeue/Push/Pop, різновиди списків і черг).
2. Map-Reduce Chunking (без перекриття): Впорався найгірше для технічної теми. Через відсутність перекриття та подвійне узагальнення на етапах Map і Reduce було втрачено точну математичну деталізацію (O-нотацію), а підсумок перетворився на стислу загальну анотацію.
3. Sliding Window (з перекриттям): Показав високу ефективність. Завдяки overlap у 35 слів модель не розірвала думки на межах розрізів і зберегла ключові показники складності та операції. Це найкраще рішення для великих технічних документів, які не поміщаються в один промпт.

Завдання 5.(2) Робота з великим контекстом

Обрати великий текстовий документ:

• текстову частину PDF-документа.

Розмір тексту повинен бути достатнім для демонстрації проблеми роботи з великим контекстом.

5.1. Робота з текстом без chunking

Передати текст мовній моделі одним запитом та виконати одну з задач:

• створення узагальнення;

• виділення ключових ідей;

• відповіді на запитання за текстом.

Зберегти результат.

1) Текст prompt:

```python
import pypdf
from google import genai
import os
client = genai.Client(api_key="API_KEY")
```

```python
MODEL = "gemini-3.5-flash-lite"

script_dir = os.path.dirname(os.path.abspath(__file__))
pdf_path = os.path.join(script_dir, "КГ.pdf")

reader = pypdf.PdfReader(pdf_path)

extracted_text = ""
for page in reader.pages[:20]:
    extracted_text += page.extract_text() + "\n"

print(f"Усього зчитано слів із PDF: {len(extracted_text.split())}")

prompt_direct = f"Проаналізуй наданий текст та зроби його детальне та точно структуроване узагальнення:\n\n{extracted_text}"

try:
    interaction_direct = client.interactions.create(
        model=MODEL,
        input=prompt_direct
    )
    direct_result = interaction_direct.output_text.strip()
    print("\n5.1 ОБРОБКА БЕЗ CHUNKING (DIRECT PROCESSING)")
    print(direct_result)
except Exception as e:
    direct_result = f"Помилка Direct Processing (можливо, перевищено ліміт контексту): {e}"
    print(direct_result)
```

2) Отримана відповідь:

```
Усього зчитано слів із PDF: 5032

5.1 ОБРОБКА БЕЗ CHUNKING (DIRECT PROCESSING)
Ось детальне та точно структуроване узагальнення змісту наданого навчального посібника В. Г. Маценка «Обчислювальна геометрія та комп'ютерна графіка» (2023).

---

# УЗАГАЛЬНЕННЯ ЗМІСТУ ПОСІБНИКА

Книга є фундаментальним навчальним виданням, яке охоплює теоретичні основи та практичні алгоритми обчислювальної геометрії та комп'ютерної графіки. Посібник роз
ділений на дві основні частини: **теоретичну** (16 розділів) та **практичну** (лабораторні роботи, проєкти, список літератури та додатки).

---

## ЧАСТИНА I. ТЕОРЕТИЧНІ ОСНОВИ ТА АЛГОРИТМИ

### 1. Математичний фундамент
*   **Аналітична геометрія (Розділ 1):** Основи векторного аналізу на площині та в просторі (додавання, скалярний, псевдоскалярний/косий, векторний та мішаний д
обутки). Моделі прямих ліній (неявне, явне, нормальне та параметричне рівняння), криві другого порядку, системи координат.
*   **Основні алгоритми комп'ютерної геометрії (Розділ 9):** Тести орієнтації (нормалі, положення точки відносно прямої/полігона), тести опуклості (полігонів і
поліедрів), тести перетину та алгоритми відсікання (Сазерленда–Коена, Кіруса–Бека, Вейлера–Азертона). Побудова опуклих оболонок (метод загортання подарунка, обх
ід Грехема) та методи тріангуляції (устаткування Делоне, діаграма Вороного).

### 2. Вступ і апаратне забезпечення комп'ютерної графіки
*   **Загальні поняття (Розділ 2):** Історія розвитку, простіші та математичні моделі графічних об'єктів сцен, сфери застосування.
*   **Технічне та програмне забезпечення (Розділ 3):** Архітектура обчислювальних засобів, графічні підсистеми, пристрої введення (сканери, дигітайзери) та виве
дення (монітори, принтери). Класифікація видів графіки (растрова, векторна, фрактальна, тривимірна) та графічні формати файлів. Масштабування та дискретизація.
```

```
### 3. Колір і колірні моделі
*   **Природа кольору та колірні простори (Розділ 4):** Фізичні основи сприйняття кольору, баланс, кодування та палітри. Детальний огляд моделей:
    *   Адитивна модель **RGB** (для екранів).
    *   Субтрактивні моделі **CMY / CMYK** (для друку).
    *   Суб’єктивна модель **HSB (HSV)**.
    Оптимальне поєднання кольорів при побудові зображень.

### 4. Растрова графіка та геометричне моделювання
*   **Растрові алгоритми (Розділи 5–6):** Числові та інкрементні алгоритми генерації ліній, кіл і еліпсів (алгоритм Брезенхема, Жордана). Алгоритми зафарбовування та заповнення областей (рекурсивні, постричкові із затравкою, за критерієм парності, YX-алгоритм, робота з текстурами).
*   **Криві та поверхні (Розділи 7–8):** Поліноміальна інтерполяція (Лагранж, сплайни), згладжуючі кубічні сплайни, криві Безьє (властивості, геометричні алгоритми), B-сплайни, криві Ерміта та TCB-сплайни. Математичні моделі поверхонь (білінійні, лінійчасті, поверхні Безьє, B-сплайнові поверхні).
*   **Фрактальна графіка (Розділ 10):** Поняття фрактала. Конструктивні фрактали (крива Коха, гілка папороті, дендрити, зіркові фрактали). Динамічні фрактали (множини Жуліа, Мандельброта, фрактали Ньютона) та їх застосування.

### 5. 2D/3D-графіка, перетворення та візуалізація
*   **Афінні перетворення (Розділ 11):** Математичне моделювання перетворень на площині та в просторі (паралельне перенесення, масштабування, обертання, відбиття).
*   **Проєкції (Розділ 12):** Класифікація проєкцій. Ортографічні, аксонометричні, косокутні та перспективні проєкції (одно-, дво- та триточкові).
*   **Системи координат (Розділ 13):** Видові перетворення, перспективне проєктування та відображення у вікно виведення (viewport).
*   **Усунення невидимих ліній і граней (Розділ 14):** Алгоритм поточного горизонту, алгоритм Робертса, метод Z-буфера, відсікання нелицьових граней, метод Варнока та алгоритм художника.
*   **Зафарбовування видимих поверхонь (Розділ 15):** Моделі відбиття світла (дзеркальне, дифузне), обчислення нормалей. Методи зафарбовування: з постійною інтенсивністю, метод Гуро, метод Фонга. Основи трасування променів (Ray Tracing).

### 6. Програмування графіки
*   **OpenGL (Розділ 16):** Основні поняття, робота з вершинами та примітивами, перетворення координат і проєкцій, виведення 3D-об'єктів, визначення видимих поверхонь, моделювання освітлення та робота з областю виведення.
```

```
## ЧАСТИНА II. ПРАКТИЧНИЙ БЛОК

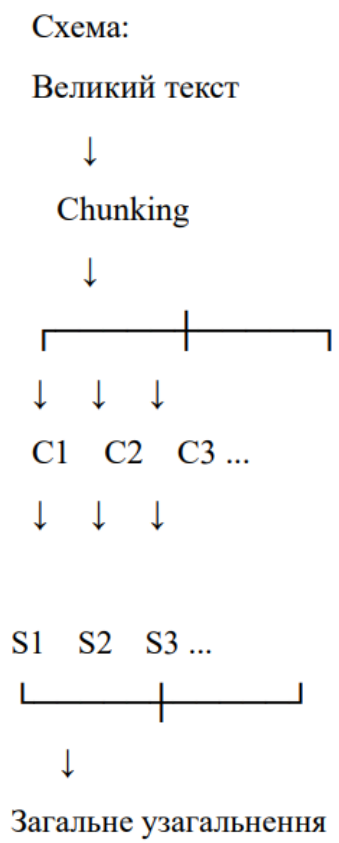
*   **Лабораторний практикум:** Містить завдання для закріплення матеріалу, поділені на чотири блоки:
    1. *Вступні завдання.*
    2. *Робота з фракталами.*
    3. *Растрові алгоритми, сплайнові криві, алгоритми відсікання.*
    4. *Візуалізація тривимірних об'єктів.*
*   **Комп’ютерні проєкти:** Комплексні завдання для самостійної розробки програмного забезпечення.
*   **Допоміжні матеріали:** Список літератури (46 джерел) та додатки.

---

### Цільова аудиторія
Посібник орієнтований на студентів вищих навчальних закладів денної та заочної форм навчання, які навчаються за спеціальностями **«Прикладна математика»**, **«Комп’ютерні науки»**, **«Системний аналіз»**, а також для аспірантів, викладачів та програмістів-розробників графічних застосунків.
```

5.2. Реалізація chunking

Розділити текст на фрагменти (chunks).



1) Текст prompt:

```
words = extracted_text.split()
chunk_size = 500
chunks = [" ".join(words[i:i + chunk_size]) for i in range(0, len(words),
chunk_size)]

print(f"\n5.2 CHUNKING (MAP-REDUCE). Текст розділено на {len(chunks)} chunks")

summaries_chunking = []
for idx, chunk in enumerate(chunks, 1):
    prompt_map = f"Зроби коротке та точне узагальнення цього фрагмента книги
(Chunk {idx}):\n\n{chunk}"
    interaction_map = client.interactions.create(model=MODEL, input=prompt_map)
    summary = interaction_map.output_text.strip()
    summaries_chunking.append(summary)
    print(f"\nSummary for Chunk {idx}:\n{summary[:150]}...")

combined_chunking = "\n\n".join([f"Частина {i+1}: {s}" for i, s in
enumerate(summaries_chunking)])
prompt_reduce_chunking = f"На основі наступних часткових узагальнень сформулюй
один цілісний підсумковий висновок для всього документа:\n\n{combined_chunking}"

interaction_reduce_chunking = client.interactions.create(
    model=MODEL,
    input=prompt_reduce_chunking
)

chunking_result = interaction_reduce_chunking.output_text.strip()
print(f"\n>>> ФІНАЛЬНИЙ РЕЗУЛЬТАТ (MAP-REDUCE CHUNKING):\n{chunking_result}")
```

2) Отримана відповідь:

```
5.2 CHUNKING (MAP-REDUCE). Текст розділено на 11 chunks

Summary for Chunk 1:
Це вихідні дані, зміст та початок навчального посібника В. Г. Маценка «Обчислювальна геометрія та комп’ютерна графіка» (ЧНУ, 2023).

У фрагменті пода...

Summary for Chunk 2:
**Узагальнення фрагмента (розділи 4–10 комп’ютерної графіки):**

Фрагмент охоплює фундаментальні алгоритми та математичні моделі комп'ютерної графіки ...

Summary for Chunk 3:
**Узагальнення фрагмента:**

У фрагменті наведено зміст другої половини книги (розділи 10–16), присвяченої фракталам, 2D/3D-перетворенням, моделюванню...

Summary for Chunk 4:
**Узагальнення фрагмента:**

Текст присвячений **обчислювальній геометрії** та **комп’ютерній графіці** як взаємопов’язаним і вкрай важливим дисциплін...

Summary for Chunk 5:
**Коротке та точне узагальнення:**

Цей фрагмент є передмовою до навчального посібника з курсу **«Обчислювальна геометрія та комп’ютерна графіка»**.
...

Summary for Chunk 6:
**Узагальнення фрагмента (Chunk 6):**
```

```
Текст присвячений елементам векторного аналізу та їх застосуванню в геометрії. У ньому розглядаються:
* **Основ...

Summary for Chunk 7:
**Узагальнення фрагмента:**

Текст присвячено поняттю **псевдоскалярного (косого) добутку векторів** на площині та його застосуванню в обчислювальній ...

Summary for Chunk 8:
**Узагальнення фрагмента (Chunk 8):**

1. **Обчислення кута між векторами:** Через тангенс (`tg ϕ`) та скалярний добуток (знак якого визначає гостроту...

Summary for Chunk 9:
Ось коротке та точне узагальнення фрагмента:

У тексті розглядаються базові поняття аналітичної геометрії та комп'ютерної графіки:
1. **Мішаний добуто...

Summary for Chunk 10:
Ось коротке та точне узагальнення фрагмента:

У фрагменті розглядаються базові геометричні та векторні рівняння на площині:
1. **Параметричне рівняння...

Summary for Chunk 11:
**Узагальнення:**

У фрагменті наведено рівняння шуканої прямої та зазначено одну з найважливіших задач геометрії на площині — знаходження точки перет...
```

```
>>> ФІНАЛЬНИЙ РЕЗУЛЬТАТ (MAP-REDUCE CHUNKING):
Ось один цілісний підсумковий висновок для всього документа:

**Цілісний підсумковий висновок:**

Документ є комплексним оглядом навчального посібника В. Г. Маценка «Обчислювальна геометрія та комп’ютерна графіка» (ЧНУ, 2023), який охоплює весь спектр тем від фундаментального математичного апарату до практичного програмування графічних систем.

Видання побудоване за логікою від теорії до практики:
1. **Математичний і геометричний фундамент:** детально розглядається векторна алгебра (скалярний, косий, векторний та мішаний добутки) та аналітична геометрія на площині (різні форми рівнянь прямих, взаємне розташування об'єктів), які є основою для будь-яких обчислень у графіці.
2. **Комп’ютерна геометрія:** охоплює алгоритми орієнтації, перетину об'єктів, відсікання, побудови опуклих оболонок, тріангуляції та діаграм Вороного.
3. **Комп’ютерна графіка та візуалізація:** розглядає апаратне та програмне забезпечення, колірні моделі, растрові алгоритми, роботу з кривими і поверхнями (сплайни Безьє, B-сплайни), фрактали, 2D/3D-перетворення, проекції, а також алгоритми видалення невидимих ліній і зафарбовування.
4. **Практична реалізація:** книга поєднує теорію з програмуванням графіки в середовищі OpenGL та містить завдання до лабораторних робіт, що робить її повноцінним навчально-методичним посібником для студентів математичних і комп’ютерних спеціальностей.
```

5.2 (Додаткове завдання). Реалізувати sliding window та порівняти його з простим поділом тексту на окремі chunks.

1) Текст prompt:

```python
window_size = 500
overlap = 100
step = window_size - overlap
sliding_chunks = []

for i in range(0, len(words), step):
    chunk_words = words[i:i + window_size]
    sliding_chunks.append(" ".join(chunk_words))
    if i + window_size >= len(words):
        break

print(f"\nSLIDING WINDOW. Текст розділено на {len(sliding_chunks)} sliding chunks")

summaries_sliding = []
for idx, chunk in enumerate(sliding_chunks, 1):
    prompt_sliding = f"Зроби узагальнення цього фрагмента книги з урахуванням суміжного контексту:\n\n{chunk}"
```

```
    interaction_sliding = client.interactions.create(model=MODEL,
input=prompt_sliding)
    summary = interaction_sliding.output_text.strip()
    summaries_sliding.append(summary)

combined_sliding = "\n\n".join([f"Фрагмент {i+1}: {s}" for i, s in
enumerate(summaries_sliding)])
prompt_reduce_sliding = f"На основі наступних часткових узагальнень (зроблених із
перекриттям контексту) сформулюй остаточне підсумкове
узагальнення:\n\n{combined_sliding}"

interaction_reduce_sliding = client.interactions.create(
    model=MODEL,
    input=prompt_reduce_sliding
)

sliding_result = interaction_reduce_sliding.output_text.strip()
print(f"\n>>> ФІНАЛЬНИЙ РЕЗУЛЬТАТ (SLIDING WINDOW):\n{sliding_result}")
```

2) Отримана відповідь:

```
SLIDING WINDOW. Текст розділено на 13 sliding chunks

>>> ФІНАЛЬНИЙ РЕЗУЛЬТАТ (SLIDING WINDOW):
Ось остаточне підсумкове узагальнення, яке інтегрує та структурує всю інформацію із наведених часткових фрагментів:

---

# Підсумкове узагальнення: Навчальний посібник В. Г. Маценка «Обчислювальна геометрія та комп’ютерна графіка» (Чернівці, 2023)

### **1. Загальна характеристика видання**
Посібник підготовлено для студентів закладів вищої освіти, які навчаються за математичними та IT-спеціальностями («Комп’ютерні науки», «Прикладна математика», « Системний аналіз»). Видання базується на багаторічному викладацькому досвіді автора та складається з двох взаємопов’язаних частин:
* **Теоретична:** фундаментальні математичні основи, апаратне та програмне забезпечення, базові та складні алгоритми комп'ютерної графіки й геометрії.
* **Практична:** лабораторний практикум, завдання для комп'ютерних проєктів, вправи для самоперевірки та контрольні питання після кожної теми.

---

### **2. Структура та зміст посібника (охоплені розділи)**

#### **Блок I: Математичний фундамент (Аналітична геометрія та векторний аналіз)**
Вивчення комп'ютерної графіки базується на строгій математичній базі (правостороння декартова система координат):
* **Векторна алгебра на площині та у просторі:** операції з векторами, колінеарність, скалярний, векторний (косий) та мішаний добутки. Їх геометричний зміст використовується для обчислення довжин, кутів, орієнтованих площ трикутників/паралелограмів, об'ємів, а також для перевірки компланарності та взаємного розташування точок (поворот за/проти годинникової стрілки).
* **Моделі прямої лінії:** загальне (неявне), з кутовим коефіцієнтом, нормальне та параметричне рівняння прямої. Особлива увага приділяється параметричному заданню, що є критично важливим для опису променів і відрізків у програмуванні, а також побудові бісектрис і перпендикулярів.

#### **Блок II: Вступ, апаратне забезпечення та види графіки**
* Історичний огляд, сфери застосування комп’ютерної графіки та класифікація програмно-апаратних засобів (графічні станції, відеосистеми, мови програмування графіки, САПР).
* Детальна класифікація графіки: растрова, векторна, фрактальна та тривимірна (3D), а також огляд стандартних файлових форматів.
```

```
#### **Блок III: Теорія кольору**
* Фізичні та психологічні основи сприйняття кольору.
* Вивчення колірних моделей: адитивної (**RGB**, для екранів), субтрактивної (**CMY/CMYK**, для друку) та суб'єктивної (**HSB/HSV**, орієнтованої на сприйняття людиною), а також питання забезпечення колірного балансу.

#### **Блок IV: Растрові алгоритми та робота з пікселями**
* **Генерування примітивів:** алгоритми побудови відрізків, кіл та еліпсів (числові методи, алгоритми Брезенхема).
* **Зафарбовування:** рекурсивні та пострічкові алгоритми з затравкою, заповнення за парністю, обробка полігонів ($YX$-алгоритм), накладання текстур.

#### **Блок V: Математичне моделювання кривих і поверхонь**
* **Криві:** інтерполяція та згладжування (поліномами Лагранжа, сплайни, криві Безьє, B-сплайни, криві Ерміта, TCB-сплайни).
* **Поверхні:** перехід до 3D — білінійні, лінійчасті, бікубічні сплайнові поверхні, поверхні Безьє та B-сплайни.

#### **Блок VI: Обчислювальна геометрія та просторові алгоритми**
* Тести на орієнтацію та приналежність точок фігурам (променевий і кутовий тести).
* Перетини прямих/відрізків, алгоритми відсікання (Сазерленда–Коена, Кіруса–Бека, Вейлера–Азертона).
* Побудова опуклих оболонок («загортання подарунка», алгоритм Грехема), тріангуляція Делоне та діаграми Вороного.

#### **Блок VII: Фрактальна геометрія**
* Дослідження природи фракталів: конструктивні (крива Коха, дендрити, папороть Барнслі) та динамічні фрактали (множини Мандельброта, Жуліа, фрактали Ньютона).

#### **Блок VIII: Геометричні перетворення та проєктування**
* Афінні перетворення у 2D та 3D (перенесення, масштабування, обертання).
* Системи координат та поетапні перетворення (видові перетворення, відображення у вікно виведення — viewport).
* Проєкції: ортографічні, аксонометричні, косокутні та перспективні (одно-, дво- і триточкові).

#### **Блок IX: Рендеринг, видимість та освітлення**
* **Усунення невидимих ліній і граней:** алгоритми Робертса, Z-буфера, поточного горизонту, художника, метод Варнока.
* **Освітлення та зафарбовування:** моделі відбиття світла (дифузне, дзеркальне), розрахунок нормалей, методи інтерполяції зафарбовування (Гуро, Фонг) та основи трасування променів (Ray Tracing).
```

```
#### **Блок X: Практична реалізація**
* Програмування тривимірної графіки за допомогою промислового стандарту **OpenGL** (робота з вершинами, примітивами, конвеєром перетворень та освітленням).

---

### **Загальний висновок**
Навчальний посібник В. Г. Маценка є комплексним академічним курсом, що охоплює весь цикл створення графічного контенту – від фундаментального математичного апар
ату та алгоритмів обчислювальної геометрії до низькорівневих растрових завдань, моделювання кривих/поверхонь, методів рендерингу (освітлення та видимості) та їх
 практичної реалізації на базі сучасних графічних бібліотек (OpenGL). Книга орієнтована на підготовку кваліфікованих розробників графічних двигунів, САПР (CAD),
 ГІС та ігрових систем.
```

5.3. Порівняння результатів

Порівняти:

1. результат роботи з текстом без chunking;

2. результат роботи з текстом після chunking.

3. результат роботи з текстом після sliding window.

Оцінити:

• повноту;

• точність;

• збереження важливої інформації;

• структурованість;

• наявність пропущеної інформації.

Сформулювати висновок щодо доцільності використання chunking.

1) Текст prompt:

```
prompt_compare = f"""
Проаналізуй та порівняй три підходи до узагальнення навчального посібника:

1. Пряма обробка без chunking (Direct):
{direct_result}

2. Метод Chunking (Map-Reduce без перекриття):
{chunking_result}

3. Метод Sliding Window (з перекриттям контексту):
{sliding_result}

Оціни їх за наступними критеріями:
- Повнота охоплення матеріалу та точність
- Збереження важливих фактів і деталей
- Структурованість підсумкового тексту
- Виявлення пропущеної інформації

Сформулюй чіткий висновок щодо доцільності використання кожного методу під час
роботи з великими документами.
"""

interaction_compare = client.interactions.create(
```

```
    model=MODEL,
    input=prompt_compare
)

print("\n5.3 ПОРІВНЯЛЬНИЙ АНАЛІЗ РЕЗУЛЬТАТІВ")
print(interaction_compare.output_text.strip())
```

2) Отримана відповідь:



```
5.3 ПОРІВНЯЛЬНИЙ АНАЛІЗ РЕЗУЛЬТАТІВ
Нижче наведено порівняльний аналіз трьох підходів до узагальнення навчального посібника за заданими критеріями, а також загальний висновок щодо доцільності їх з
астосування для роботи з великими документами.

---

### Порівняльний аналіз підходів

#### 1. Пряма обробка без chunking (Direct)
*   **Повнота охоплення та точність:** **Висока.** Оскільки весь документ обробляється одночасно (за умови, що він поміщається в контекстне вікно моделі), модел
ь має повну картину з самого початку. Це забезпечує високу точність у збереженні логічних зв'язків.
*   **Збереження важливих фактів і деталей:** **Добре.** Добре збережено назви конкретних алгоритмів (Брезенхема, Сазерленда–Коена, алгоритм Гуро/Фонга тощо) та
 розподіл на теоретичну й практичну частини.
*   **Структурованість:** **Відмінна.** Текст має чітку ієрархію (заголовки, списки), легко читається завдяки візуальному поділу на тематичні блоки.
*   **Виявлення пропущеної інформації:** **Обмежена.** Модель спирається лише на те, що подано в огляді, проте ризик «забути» частину тексту мінімальний, оскіль
ки вона бачить увесь документ цілком.

#### 2. Метод Chunking (Map-Reduce без перекриття)
*   **Повнота охоплення та точність:** **Низька / Середня.** Підхід дає надто лаконічний (узагальнений) результат. Через відсутність перекриття між частинами те
ксту на етапі обробки втрачаються деталі та специфічні назви алгоритмів.
*   **Збереження важливих фактів і деталей:** **Слабке.** Деталі зникли на користь загальних фраз (наприклад, перелік конкретних колірних моделей чи алгоритмів
відсікання зведено до загального опису).
*   **Структурованість:** **Добра.** Текст структурований логічно (нумеровані пункти), але містить занадто мало інформації для посібника такого обсягу.
*   **Виявлення пропущеної інформації:** **Незадовільна.** Метод "Map-Reduce" без перекриття на етапі збирання частіше «стирає» унікальні факти, вважаючи їх мал
означущими для загальної картини.

#### 3. Метод Sliding Window (з перекриттям контексту)
*   **Повнота охоплення та точність:** **Максимальна.** Завдяки перекриттю контексту модель не втрачає зв'язки між суміжними розділами. Отримано найповніший і н
айдетальніший перелік усіх тем, розділів і математичних аспектів.
*   **Збереження важливих фактів і деталей:** **Відмінне.** Збережено навіть нюанси на кшталт конкретних типів сплайнів (TCB-сплайни), моделей освітлення, метод
ів тріангуляції (Делоне, діаграми Вороного) та цільової аудиторії.
*   **Структурованість:** **Чудова.** Використано глибоку ієрархію (Блоки I–X), яка детально розкриває зміст кожної частини книги.
*   **Виявлення пропущеної інформації:** **Найвища.** Метод «ковзного вікна» дозволяє виявити й зафіксувати переходи між темами, запобігаючи «сліпим зонам» на с
тиках фрагментів тексту.
```



```
### Загальний висновок щодо доцільності використання методів

1.  **Пряма обробка (Direct)** є **ідеальним вибором**, якщо розмір документа не перевищує ліміт контекстного вікна моделі (що є стандартним для сучасних LLM з
великим контекстом, наприклад, Claude 3.5 Sonnet або GPT-4o). Вона забезпечує найкращий баланс між деталізацією та цілісністю без зайвих витрат на пайплайни.
2.  **Метод Chunking (Map-Reduce без перекриття)** є **малоефективним** для роботи з навчальною чи технічною літературою. Він призводить до надмірного стиснення
 («галюцинації узагальнення»), внаслідок чого втрачаються критично важливі деталі (назви алгоритмів, термінологія). Його варто використовувати лише тоді, коли п
отрібна ультракоротка вижимка (exec-summary) без технічних деталей.
3.  **Метод Sliding Window (з перекриттям контексту)** є **найкращим рішенням для дуже великих документів** (книг, мануалів, досліджень), які фізично не поміщаю
ться в контекстне вікно або вимагають поетапного глибокого аналізу. Перекриття контексту нівелює головний недолік звичайного chunking – втрату зв'язків на межах
 розділів, що дозволяє створити максимально деталізоване, структуроване та точне узагальнення.
```

Завдання 6. Дослідження параметрів генерації

Обрати одну задачу, для якої результат генерації можна порівнювати.

Наприклад:

• написання короткого тексту;
• генерація опису;
• створення запитань;
• генерація ідей;
• класифікація тексту.

Завдання 6.1. Виконати одну й ту саму задачу з різними значеннями параметра temperature:

• 0;
• 0.5;
• 1

Для кожного значення зберегти результат.

Порівняти відповіді за такими критеріями:

• точність;

• креативність;
• різноманітність;
• стабільність результату.
1) Текст prompt:

```
from google import genai

client = genai.Client(api_key="API_KEY")
MODEL = "gemini-3.5-flash-lite"

prompt_creative = "Придумай 3 оригінальні назви та короткі концепти для стартапу
у сфері штучного інтелекту для навчання."

temperatures = [0.0, 0.5, 1.0]

print("6.1 ДОСЛІДЖЕННЯ ПАРАМЕТРА TEMPERATURE")
for temp in temperatures:
    print(f"\nTemperature = {temp}")
    interaction_temp = client.interactions.create(
        model=MODEL,
        input=prompt_creative,
        generation_config={
            "temperature": temp
        }
    )
    print(interaction_temp.output_text.strip())
```

2) Отримана відповідь:

```
6.1 ДОСЛІДЖЕННЯ ПАРАМЕТРА TEMPERATURE

Temperature = 0.0
О ось 3 оригінальні назви та концепти для стартапу у сфері освітнього штучного інтелекту:

### 1. Назва: **Synapse.me** (Сінапс мі)
*   **Концепт:** Нейромережа-компаньйон, яка створює унікальну "картку пам'яті" для кожного студента, імітуючи роботу синапсів головного мозку. ШІ аналізує, як саме користувач засвоює інформацію (візуально, через аудіо, аналогії чи практичні задачі), і на льоту переписує будь-який навчальний матеріал (від квантової фіз ики до історії мистецтв) під цей когнітивний профіль. Замість загальних лекцій студент отримує пояснення складних тем через свої особисті захоплення (наприклад, вивчення економіки через механіки улюбленої відеогри).

### 2. Назва: **Aura.Edu** (Аура Едюкейшн)
*   **Концепт:** Емоційний репетитор на базі ШІ, що дбає про ментальний стан під час навчання. Використовуючи камеру та мікрофон (за згодою користувача), ШІ від стежує рівень стресу, зосередженості та втоми за мімікою, тоном голосу та швидкістю друку. Якщо система бачить розгубленість — вона м'яко спрощує матеріал або п ропонує 2-хвилинну дихальну паузу. Якщо студент нудьгує — підвищує рівень складності та додає елементи челенджу. Це навчання, яке адаптується не лише під ваш ін телект, а й під ваш настрій.

### 3. Назва: **Socratic.AI** (Сократік Ей-І)
*   **Концепт:** Анти-ГПТ для освіти. У епоху, коли звичайні чат-боти дають готові відповіді на домашні завдання, цей ШІ навмисно *не дає відповідей*. Застосову ючи сократівський метод викладання, він спілкується зі студентом виключно через наводящі запитання. ШІ м'яко веде учня до самостійного вирішення проблеми, розби ваючи складну задачу на мікрокроки. Це розвиває критичне мислення та не дозволяє списувати, перетворюючи ШІ на справжнього мудрого ментора.

Temperature = 0.5
О ось 3 оригінальні назви та концепти для стартапу у сфері освітнього штучного інтелекту:

### 1. Назва: **Syllapulse** (від англ. *Syllabus* — навчальна програма + *Pulse* — пульс)

*   **Концепт:** AI-ментор, який «відчуває ритм» життя студента та динамічно адаптує навчальний план у реальному часі.
*   **Як це працює:** Якщо студент захворів, завантажений на роботі чи, навпаки, засвоїв матеріал швидше, ніж планувалося, алгоритм автоматично перекроює дедлай ни, перерозподіляє навантаження та генерує мікро-уроки (капсули знань по 5 хвилин), щоб людина не відстала і не вигоріла. Це не просто розклад, це живий організ м навчального процесу.
```

```
### 2. Назва: **EchoClass** (від *Echo* — відлуння/повторення + *Class* — клас)

*   **Концепт:** Платформа «машини часу» для вивчення складних концептів через голос історичних особистостей або вигаданих персонажів.
*   **Як це працює:** ШІ аналізує підручник чи лекцію і перетворює її на інтерактивний діалог-подкаст. Наприклад, квантову фізику вам може «пояснювати» Альберт Ейнштейн, а правила економіки — Воррен Баффет. Студент може сперечатися з ними голосом, ставити уточнювальні запитання, а ШІ генерує відповіді унікальним стилем та голосом обраного персонажа, роблячи навчання захопливим серіалом.

---

### 3. Назва: **MindWeave** (від *Mind* — розум + *Weave* — ткати/плетіння)

*   **Концепт:** Візуальний ШІ-редактор, який будує персональну «картку знань» користувача, зв'язуючи нові предмети з його реальними інтересами.
*   **Як це працює:** Багато хто вчить речі ізольовано і забуває їх. *MindWeave* сканує те, що ви любите (фільми, відеоігри, спорт, хобі), і пояснює нові академічні теми через ці аналогії. Наприклад, вивчаєте програмування? ШІ покаже логіку кодів через механіки улюбленої гри користувача. Платформа буквально «вплітає» нові знання у вже наявну нейронну мережу досвіду людини.

Temperature = 1.0
О ось 3 оригінальні назви та концепти для стартапу у сфері освітнього штучного інтелекту:

### 1. Назва: **NeuroScribe** (НейроСкрайб)
* **Слоган:** Твій персональний архітектор знань.
* **Концепт:** Це ШІ-платформа для глибокого засвоєння інформації, яка перетворює будь-які складні матеріали (лонгріди, наукові статті, відеолекції) на інтерактивні художні історії або комікси, адаптовані під стиль мислення користувача. Якщо ви візуал — ШІ створить сюжетну графічну новелу для вивчення квантової фізики. Якщо аудіал — захопливий поддіст-розслідування. Платформа аналізує, як ваш мозок найкраще засвоює інформацію, і динамічно переписує навчальний контент у реальному часі.

### 2. Назва: **Socratic.AI** (Сократік.ШІ)
* **Слоган:** Навчайся, ставлячи правильні запитання.
* **Концепт:** Антитеза традиційним ШІ-репетиторам, які просто дають готові відповіді. Цей стартап базується на сократівському методі навчання (метод діалогу). Коли учень вводить завдання або питання, ШІ *ніколи* не дає рішення. Натомість він стає допитливим віртуальним співрозмовником, який м'яко скеровує студента серією підказок і запитань до самостійного осягнення суті. Це розвиває критичне мислення та унеможливлює просте «списання» домашньої роботи.

### 3. Назва: **EmpathyEdu** (ЕмпатіЕдю)
* **Слоган:** ШІ, який відчуває, коли ти втомлюєшся.
* **Концепт:** Платформа емоційного та адаптивного навчання. За допомогою веб-камери (аналіз мікроміміки) та швидкості введення тексту ШІ відстежує рівень когнітивного навантаження, стресу, розгубленості чи нудьги учня. Щойно система бачить, що студент «плаває» або втомився, вона автоматично змінює формат подачі матеріалу: робить коротку гейміфіковану перерву, пропонує легшу задачу для відновлення впевненості або змінює темп уроку. Це репетитор із високим рівнем емоційного інтелекту (EQ).
```

Аналіз результатів:

Точність: Найвища при temp = 0.0 та temp = 0.5. Концепти чітко відповідають EdTech-тематиці та мають зрозумілу структуру. При temp = 1.0 точність зберігається, але модель додає рекламні елементи (слогани), дещо відходивши від прямого опису ідеї.

Креативність: Зростає із збільшенням температури. При 0.0 ідеї базові та стандартні. При 0.5 модель демонструє найкращий баланс оригінальності (цікаві метафори: *Syllapulse*, *MindWeave*). При 1.0 креативність висока, але з'являється більше шаблонних образів.

Різноманітність: При temp = 0.0 та temp = 1.0 моделі прийшли практично ідентичні концепти (сократівський ШІ, емоційний репетитор через веб-камеру, персоналізація під когнітивний профіль). Найбільш оригінальними та різноманітними виявилися концепти при temp = 0.5.

Стабільність результату: При temp = 0.0 стабільність є максимальною (детермінованою) - при повторних запусках модель видаватиме практично той самий текст. З підвищенням температури до 1.0 стабільність знижується, а варіативність слів та ідей зростає.

Завдання 6.2 Дослідження параметра top-p
Повторити експеримент із декількома значеннями top-p, наприклад:
• 0.5;
• 0.8;
• 1.0.
Порівняти результати та зробити висновок про вплив параметра на генерацію.

1) Текст prompt:

```
top_ps = [0.5, 0.8, 1.0]

print("\n6.2 ДОСЛІДЖЕННЯ ПАРАМЕТРА TOP_P")
```

```python
for p in top_ps:
    print(f"\nTop_P = {p}")
    interaction_p = client.interactions.create(
        model=MODEL,
        input=prompt_creative,
        generation_config={
            "top_p": p
        }
    )
    print(interaction_p.output_text.strip())
```

2) Отримана відповідь:



```
6.2 ДОСЛІДЖЕННЯ ПАРАМЕТРА TOP_P

Top_P = 0.5
О ось 3 оригінальні назви та концепти для стартапу у сфері освітнього штучного інтелекту:

### 1. Назва: **Syllabot** (Силлабот)
*   **Концепт:** AI-наставник із проєктування персональних освітніх траєкторій (силлабусів).
*   **Як це працює:** Замість стандартних онлайн-курсів, користувач завантажує свою кінцеву мету (наприклад: «Вивчити Python для створення телеграм-ботів за 2 м
ісяці, маючи 5 годин на тиждень»). ШІ сканує мільйони джерел (YouTube, статті, GitHub, книги) і створює живий, динамічний план навчання. Якщо студент відстає аб
о, навпаки, швидко опановує тему, бот на льоту перекроює розклад, підбираючи формат матеріалу під стиль засвоєння інформації конкретною людиною (візуал, аудіо,
практика).

### 2. Назва: **EchoClass** (ЕхоКлас)
*   **Концепт:** Платформа «живої» імітації діалогів для практичного навчання.
*   **Як це працює:** Найважче в навчанні — перехід від теорії до практики. EchoClass генерує AI-персонажів для моделювання реальних ситуацій. Хочете вивчити ін
озему мову? Ваш співрозмовник — шкідливий турист в паризькому кафе. Вивчаєте маркетинг? Ваш «опонент» — вибагливий клієнт, якому треба продати продукт. ШІ ана
лізує ваші відповіді в реальному часі, вказує на логічні чи фактичні помилки та дає зворотний зв'язок на основі психології переговорів.

### 3. Назва: **MnemoPulse** (МнемоПульс)
*   **Концепт:** ШІ-оптимізатор повторення матеріалу на основі біометричних даних.
*   **Як це працює:** Додаток синхронізується з розумним годинником або фітнес-браслетом користувача. Аналізуючи рівень стресу, якість сну та пульс, алгоритм ви
значає пікові хвилини когнітивної активності протягом дня. Саме в ці моменти «пульсу» система надсилає короткі мікротести для повторення пройденого матеріалу (і
нтервальне повторення). Це дозволяє закріплювати знання в довгостроковій пам'яті втричі швидше, не перенапружуючи мозок у періоди втоми.

Top_P = 0.8
О ось 3 оригінальні назви та концепти для стартапів у сфері EdTech на базі штучного інтелекту:

### 1. Назва: **Synapse.me**
* **Слоган:** Твоя нейромережа для знань.
* **Концепт:** Це персональний ШІ-ментор, який адаптує процес навчання під унікальний «цифровий слід» і стиль мислення користувача. Замість стандартних курсів д
одаток аналізує, як саме людина засвоює інформацію (через подкасти, лонгріди, схеми чи короткі відео), і на льоту генерує мікро-уроки, комбінуючи світові бази з
нань під конкретну людину. Якщо ви візуал — ШІ перетворить складну теорію квантової фізики на інтерактивний комікс; якщо аудіал — на захопливий діалог-подкаст.

### 2. Назва: **AuraEd**
* **Слоган:** Емоційний інтелект твого навчання.
* **Концепт:** Платформа фокусується не лише на академічних успіхах, а на психоемоційному стані учня під час навчання. За допомогою комп'ютерного зору (web-каме
ра) та аналізу паттернів введення тексту ШІ визначає рівень стресу, втоми чи нудьги. Якщо система бачить, що студент «плаває» або вигоряє, вона автоматично змін
ює складність матеріалу, пропонує 5-хвилинну гейміфіковану паузу або перемикає формат подачі на більш розважальний.

### 3. Назва: **Socratic.ai**
* **Слоган:** Навчайся через запитання, а не відповіді.
* **Концепт:** Антидот до сучасних чат-ботів, які просто дають готові рішення. Цей ШІ побудований на сократівському методі навчання. Коли студент завантажує туд
и складне домашнє завдання чи робочу проблему, бот **ніколи** не дає правильної відповіді. Натомість він стає допитливим партнером по дискусії, який наводить на
думку серією влучних, поступово складніших запитань. Це розвиває критичне мислення та вчить людину думати самостійно, а не просто копіювати результати роботи н
ейромереж.

Top_P = 1.0
О ось 3 оригінальні назви та концепти для стартапу у сфері освітнього штучного інтелекту, орієнтовані на сучасні тренди персоналізації та гейміфікації:

### 1. Назва: **Synapse.me** (Синапс мі)
*   **Концепт:** AI-ментор, який «зрощує» нові нейронні зв'язки у вашому мозку. Замість загальних курсів, цей додаток аналізує ваш темп мислення, професійні ціл
і та поточний рівень за допомогою коротких щоденних діалогів. Він створює гіперперсоналізовану мікро-програму навчання (по 5-10 хвилин на день), підлаштовуючи ф
ормат подачі матеріалу під ваш унікальний когнітивний стиль (візуал, аудіал, кінестетик).
*   **Фішка:** Функція «Пам'ять золотої рибки» — ШІ постійно сканує ваш прогрес і автоматично повертає забутий матеріал у вигляді інтерактивних квізів у момент
найбільшої ймовірності забування.

### 2. Назва: **PersonaProf** (Персонапроф)
*   **Концепт:** Платформа «живих» історичних та літературних персонажів для вивчення гуманітарних наук і бізнес-скітів. Хочете вивчити економіку? Вам пояснює Ї
й Адам Сміт. Потрібно зрозуміти епоху Відродження? Ви спілкуєтеся з аватаром Леонардо да Вінчі, який володіє всією повнотою історичних знань і говорить сучасною
мовою.
*   **Фішка:** ШІ-персонажі не просто видають текст, а моделюють реальні історичні чи бізнес-ситуації. Наприклад, студент може спробувати «переконати» Наполеона
змінити стратегію або пройти співбесіду з віртуальним Стівом Джобсом.

### 3. Назва: **EchoEd** (Екоед)
*   **Концепт:** ШІ-асистент для перетворення будь-якого контенту на інтерактивний навчальний всесвіт. Ви завантажуєте в систему скучну бізнес-книгу, річний зві
т компанії, подкаст або серію лекцій, а нейромережа за лічені секунди генерує з них повноцінний інтерактивний курс: з лором (сюжетом), персонажами, квізами, пра
ктичними симуляціями та нагородами.
*   **Фішка:** Перетворення пасивного споживання інформації (читання/слухання) на активну рольову гру, де кожен користувач стає головним героєм історії, що вивч
ається.
```

Аналіз результатів:

Точність: Висока на всіх рівнях. При top_p = 0.5 модель видає суворі, логічно сконцентровані та збалансовані описи без зайвого «поетичного» тексту.

Креативність: Поступово зростає від 0.5 до 1.0. Збільшення вибірки дозволяє моделі обирати менш очевидні концепції (наприклад, імітація розмови з Стівом Джобсом чи Адамом Смітом у PersonaProf при 1.0).

Різноманітність: При top_p = 0.8 та 1.0 модель часто повертається до найбільш ймовірних трьох "якорів" (*Synapse.me, Socratic.ai, AuraEd/EchoEd*). При top_p = 0.5 набір стартапів виявився абсолютно унікальним (*Syllabot, EchoClass, MnemoPulse*).

Стабільність результату: При top_p = 0.5 відповіді найбільш прогнозовані та стійкі, оскільки модель жорстко відсікає 50% малоймовірних слів. При top_p = 1.0 модель розглядає повний спектр слів, що знижує повторюваність.

Висновки:

Вплив Temperature:

- Низька температура (0.0): Забезпечує детермінованість, високу точність та логічну стабільність. Підходить для точних аналітичних задач, програмування, JSON-структур та витягу фактів.
- Середня температура (0.5): Забезпечує оптимальний баланс між логікою, точністю та оригінальністю.
- Висока температура (1.0): Максимізує креативність та варіативність, але збільшує ризик галюцинацій і знижує передбачуваність структури.

Вплив Top_P:

- Низький top_p (0.5): Звужує вибірку слів до найімовірніших (відсікає 50% «хвоста»), що робить відповідь максимально лаконічною, строгою та передбачуваною.
- Помірний top_p (0.8): Забезпечує збалансоване розширення словникового запасу (відсікає лише 20% найменш ймовірних слів), створюючи гарний асоціативний ряд без втрати чіткості.
- Максимальний top_p (1.0): Повністю знімає обмеження на вибірку слів (використовує 100% ймовірностей), що чудово підходить для глибокого брейнштормінгу та генерації нестандартних ідей.

Завдання 7. Загальне порівняння.

За результатами лабораторної роботи сформувати підсумкову таблицю:

Підхід	Якість	Повнота	Стабільність	Креативність
Простий prompt	3	3	4	2
Просунутий prompt	5	5	5	3
Декомпозиція	5	5	5	2
Chunking	3	2	4	2
Sliding Window	4	4	4	2
Temperature = 0.0	5	4	5	1
Temperature = 0.5	4	4	4	3
Temperature = 1.0	3	4	2	5

На основі результатів сформулювати висновок:

1) Техніки, що суттєво покращили якість відповіді:

1. Просунутий prompt та системні інструкції - задання чіткої ролі моделі, контексту, обмежень та бажаного формату результату повністю усуває галюцинації та «воду», роблячи відповідь максимально структурованою і точною.

2. Декомпозиція задачі (покрокове виконання): Розбиття складної задачі на декілька послідовних етапів суттєво підвищує глибину опрацювання матеріалу. Вона гарантує, що модель не пропустить важливі деталі та збереже логічний зв'язок під час генерації.

2) Коли доцільно використовувати декомпозицію:

Декомпозицію (покрокове розбиття задачі на етапи) доцільно використовувати для складних багатокомпонентних задач (наприклад, генерація тестових питань із виділенням фактів, варіантів і правильних відповідей). Вона запобігає пропускам важливих деталей моделю та забезпечує логічну послідовність виконання.

3) Коли необхідний Chunking (або Sliding Window):

Chunking є необхідним, коли розмір вхідного документу перевищує ліміт контекстного вікна LLM або коли вимагається обробка величезних масивів даних. Для великих технічних текстів найкращим виявився підхід Sliding Window (з перекриттям), оскільки він запобігає розриву слів та логічних думок на межах блоків.

4) Як параметри генерації впливають на результат:

1. temperature відповідає за рівень випадковості та фантазії: низька температура розгладжує ймовірності до єдиного найвірогіднішого слова, а висока - робить вибір слів ризикованим і креативним.

2. top_p керує обсягом словникового запасу candidate words: відсікає малоймовірні слова («хвіст»), визначаючи межі сміливості моделі.

5) Рекомендації щодо вибору параметрів під типи задач:

1. Для точних, аналітичних та програмістських задач (парсер JSON, математика, аналіз даних, витяг фактів): доцільно використовувати temperature = 0.0 та top_p = 0.5 (максимальна стабільність, детермінованість і нульовий ризик галюцинацій).

2. Для творчих задач та брейнштормінгу (генерація назв стартапів, концептів, написання сценаріїв чи віршів): доцільно використовувати temperature = 0.8…1.0 та top_p = 0.8…1.0 (максимальна варіативність, багатий асоціативний ряд та оригінальність).